

ΕΘΝΙΚΟ ΜΕΤΣΟΒΙΟ ΠΟΛΥΤΕΧΝΕΙΟ

ΣΧΟΛΗ ΕΦΑΡΜΟΣΜΕΝΩΝ ΜΑΘΗΜΑΤΙΚΩΝ ΚΑΙ ΦΥΣΙΚΩΝ ΕΠΙΣΤΗΜΩΝ

8ο ΕΞΑΜΗΝΟ

Ακ. Έτος: 2025–2026



Βάσεις Δεδομένων 2026

Ελένη Στιβακτάκη, Δημήτριος-Χρυσοβαλάντης Γαλανός,
Μιχάλης Θρασυβουλίδης

Αριθμοί Μητρώων: 09122122, 09122096, 09122014

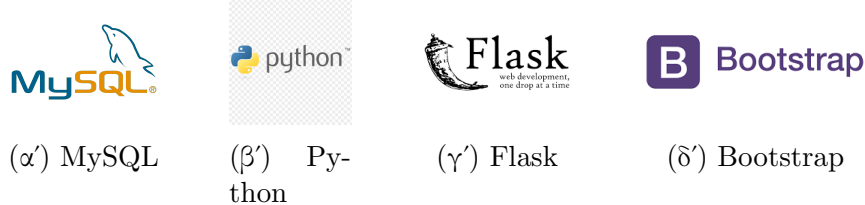
<https://github.com/ElenaStiv/ygeiopolis.git>

Ημερομηνία: 19 Μαΐου 2026

Περιεχόμενα

1	Σχεδίαση Ιστοσελίδας	3
2	Δομή Αρχείων	3
3	Οδηγίες Χειρισμού (Εγκατάσταση)	5
4	Αναγκαίοι Περιορισμοί Εκφώνησης	6
5	Υποθέσεις Υλοποίησης	8
6	Αιτιολόγηση ερωτήματος 3	11
7	Entity Relationship Diagram	11
8	Relational Diagram	12
9	SQL DDL SCRIPT	13
10	Χρήση Ευρετηρίων και Στρατηγικών Join	23
10.1	Ερώτημα 4ο	23
10.2	Ερώτημα 6ο	25

1 Σχεδίαση Ιστοσελίδας



Σχήμα 1: Web Development Stack Συστήματος

Η εφαρμογή σχεδιάστηκε στο εξής stack τεχνολογιών: MySQL (για τη σχεσιακή βάση δεδομένων), Python και Flask (για το backend), και HTML5/CSS3 με Bootstrap 5 (για τον σχεδιασμό και την προσαρμοστικότητα του Frontend - Responsive UI). (βλ. και σχήμα 1).

Το σύστημα ακολουθεί αρχιτεκτονική **Server-Side Rendering (SSR)**. Με τη χρήση του Flask framework αναπτύχθηκε ο διακομιστής (server) που διαχειρίζεται τη λογική της εφαρμογής και την απευθείας επικοινωνία με τη βάση δεδομένων. Η διεπαφή χρήστη αποδίδεται δυναμικά από τον server με τη χρήση της μηχανής προτύπων **Jinja2**, δημιουργώντας τελικά τα αρχεία HTML που αποστέλλονται και προβάλλονται στον client (στον browser του χρήστη).

Η αλληλεπίδραση του χρήστη με το σύστημα (π.χ. επιλογή βαρδιών, κλείσιμο ραντεβού, υποβολή αξιολογήσεων) πραγματοποιείται μέσω φορμών (HTML Forms) με χρήση **HTTP POST και GET requests**. Τα δεδομένα αντλούνται μέσω της βιβλιοθήκης του Flask, ελέγχονται στον server και κατόπιν εκτελούνται τα αντίστοιχα SQL Queries (INSERT, SELECT, UPDATE) στη βάση δεδομένων.

Σε ένα πραγματικό περιβάλλον παραγωγής (production), η επικοινωνία μεταξύ του client και του server θα προστατεύεται υποχρεωτικά με πρωτόκολλο HTTPS, εξασφαλίζοντας την κρυπτογράφηση και την ασφάλεια των ευαίσθητων ιατρικών και προσωπικών δεδομένων των ασθενών.

2 Δομή Αρχείων

Η αρχιτεκτονική οργάνωση του πηγαίου κώδικα της εφαρμογής ακολουθεί τη δομή που επιβάλλει το Flask framework. Ο κώδικας έχει διαχωριστεί με σαφήνεια σε επίπεδο λογικής (Backend), παρουσίασης (Frontend) και στατικών αρχείων, εξασφαλίζοντας την ευκολία στη συντήρηση και την επεκτασιμότητα του συστήματος.

Παρακάτω παρουσιάζεται η σχηματική δομή των καταλόγων (directory tree) του project:

```
ygeiopolis/
```

```
  README.md
```

```
  diagrams/
    er.pdf
    relational.pdf
```

```
sql/
  install.sql
  load.sql           # fake data
  Q1.sql, Q2.sql ...
  Q1_out.txt ...     # results from RDBMS

docs/
  report.pdf

code/
  run.py             # initiation Flask server
  routes.py
  requirements.txt
  static/            # logos, iamges
  templates/         # HTML (Jinja2) - Frontend
```

Αναλυτική Περιγραφή Αρχείων

- *run.py* / *app.py*: Αποτελεί το κεντρικό σημείο εισόδου (entry point) της εφαρμογής. Είναι υπεύθυνο για την αρχικοποίηση του Flask application, τη ρύθμιση των παραμέτρων σύνδεσης με τη βάση δεδομένων MySQL (κεντρικός διακομιστής, όνομα χρήστη, κωδικός πρόσβασης), καθώς και για την εκκίνηση του τοπικού web server.
- *routes.py*: Το αρχείο αυτό περιέχει τον "εγκέφαλο" του backend. Εδώ ορίζονται όλες οι διαδρομές (routes) της εφαρμογής μέσω Python decorators (π.χ. `@app.route`). Κάθε διαδρομή εκτελεί τη δική της επιχειρηματική λογική: ανοίγει κανάλι επικοινωνίας με τη MySQL μέσω cursors, εκτελεί τα κατάλληλα SQL Queries (SELECT για προβολή, INSERT για καταχώρηση), επεξεργάζεται τα αποτελέσματα και τα τροφοδοτεί στα αντίστοιχα αρχεία του φακέλου templates.
- *requirements.txt*: Αρχείο κειμένου που καταγράφει με ακρίβεια όλες τις εξωτερικές βιβλιοθήκες Python και τις εκδόσεις τους (όπως Flask, Flask-MySQLdb) που είναι απαραίτητες για τη σωστή εκτέλεση της εφαρμογής σε οποιοδήποτε νέο περιβάλλον.
- **Φάκελος** *static/*: Περιλαμβάνει όλα τα στατικά αρχεία που δεν αλλάζουν δυναμικά. Στον υποφάκελο *images* είναι αποθηκευμένα το λογότυπο του νοσοκομείου "Υγειόπολις", καθώς και οι φωτογραφίες του ιατρικού προσωπικού και του τεχνολογικού εξοπλισμού, οι οποίες φορτώνονται τοπικά από την εφαρμογή.
- **Φάκελος** *templates/*: Φιλοξενεί τα αρχεία HTML5 της εφαρμογής. Τα αρχεία αυτά δεν είναι στατικά, αλλά αποτελούν πρότυπα (templates) της μηχανής Jinja2 και αλληλεπιδρά πραγματικά με τη βάση μας.

- **README.md:** Ένα συνοπτικό αρχείο κειμένου που περιέχει βασικές πληροφορίες για την εργασία, τα προαπαιτούμενα εργαλεία (Python, MySQL) και οδηγίες για την εγκατάσταση και εκτέλεση της εφαρμογής.
- **Φάκελος *diagrams/*:** Περιέχει τα παραδοτέα του θεωρητικού σχεδιασμού. Το *er.pdf* αποτυπώνει το εννοιολογικό μοντέλο της βάσης, ενώ το *relational.pdf* παρουσιάζει το τελικό σχεσιακό σχήμα που προέκυψε από τη μετάφραση του εννοιολογικού.
- **Φάκελος *sql/*:** Συγκεντρώνει όλη την αλληλεπίδραση με το RDBMS (MySQL).
 - Το *install.sql* περιέχει τα ερωτήματα δημιουργίας δεδομένων (DDL - CREATE TABLE) και τους περιορισμούς ξένων κλειδιών.
 - Το *load.sql* ενσωματώνει όλες τις εντολές μαζικής εισαγωγής (INSERT) για τον εμπλουτισμό της βάσης με ρεαλιστικά δεδομένα (όπως ασθενείς, ICD-10, KEN, φάρμακα και βάρδιες).
 - Τα αρχεία *Qx.sql* και *Qx_out.txt* αντιστοιχούν στην επίλυση και την έξοδο (αποτελέσματα) των επιμέρους ερωτημάτων της εκφώνησης.
- **Φάκελος *docs/*:** Περιέχει την παρούσα αναφορά (*report.pdf*), η οποία τεκμηριώνει πλήρως την αρχιτεκτονική του συστήματος, τις επιλογές υλοποίησης και τα ζητούμενα screenshots της εφαρμογής.

Disclaimer: Για τη δημιουργία fake data, καθώς και στη παραγωγή πηγαίου κώδικα για την ιστοσελίδα έγινε χρήση γνωστού LLM.

3 Οδηγίες Χειρισμού (Εγκατάσταση)

Για να τρέξετε την εφαρμογή τοπικά στον υπολογιστή σας, παρακαλώ ακολουθήστε τα παρακάτω απλά βήματα:

1. Προαπαιτούμενα

Βεβαιωθείτε ότι έχετε εγκατεστημένα:

- *Python* (έκδοση 3.x)
- Έναν τοπικό σερβερ *MySQL* (π.χ., *XAMPP*, *WAMP*, ή αυτόνομη *MySQL*)

2. Ρύθμιση της Βάσης Δεδομένων (*MySQL*)

1. Ανοίξτε το *phpMyAdmin* (ή άλλο εργαλείο διαχείρισης *MySQL*).
2. Δημιουργήστε μια νέα, κενή βάση δεδομένων με το όνομα: *ygeiopolis* (Προτείνεται *Collation: utf8mb4_general_ci* για σωστή υποστήριξη ελληνικών).

3. Επιλέξτε τη νέα βάση και μεταβείτε στην καρτέλα 'Εισαγωγή' (*Import*).
4. Επιλέξτε το αρχείο *ygeiopolis.sql* που περιλαμβάνεται στον φάκελο και πατήστε 'Εκτέλεση' (*Go*).

3. Ρύθμιση Κωδικών Πρόσβασης στον Κώδικα

Ανοίξτε το αρχείο της εφαρμογής που διαχειρίζεται τη σύνδεση με τη βάση (π.χ., το *__init__.py* ή το *app.py*) και βρείτε τις ρυθμίσεις της *MySQL*. Βεβαιωθείτε ότι τα στοιχεία (*username*, *password*) ταιριάζουν με τα δικά σας.

Παράδειγμα:

```
app.config['MYSQL_USER'] = 'root'  
app.config['MYSQL_PASSWORD'] = ''
```

Βάλτε εδώ τον δικό σας κωδικό, αν υπάρχει.

4. Εγκατάσταση Βιβλιοθηκών (*Dependencies*)

Ανοίξτε ένα τερματικό (*CommandPrompt*, *PowerShell* ή το προτιμότερο το τερματικό του *VS Code*), πλοηγηθείτε στον φάκελο της εφαρμογής και τρέξτε την παρακάτω εντολή:

```
pip install -r requirements.txt
```

Αυτό θα εγκαταστήσει το *Flask* και τη βιβλιοθήκη επικοινωνίας με τη *MySQL*.

5. Εκτέλεση της Εφαρμογής

Στο ίδιο τερματικό, τρέξτε την εφαρμογή με την εντολή:

```
python run.py
```

Μόλις ξεκινήσει ο *server*, ανοίξτε τον *browser* της επιλογής σας και επισκεφθείτε τη διεύθυνση:
`http://127.0.0.1:5000/`

4 Αναγκαίοι Περιορισμοί Εκφώνησης

Περιορισμοί Εποπτείας (Ιεραρχία Ιατρών)

Οι ιατροί με βαθμίδα «Ειδικευόμενος» πρέπει υποχρεωτικά να έχουν επόπτη.

Οι ιατροί με βαθμίδα «Διευθυντής» δεν επιτρέπεται να έχουν επόπτη.

Απαγορεύεται η κυκλική εποπτεία (π.χ. ο ιατρός A να είναι επόπτης του B, και ο B επόπτης του A).

Περιορισμοί Χειρουργείων Επεμβάσεων

Δύο ιατρικές επεμβάσεις δεν μπορούν να πραγματοποιούνται ταυτόχρονα στον ίδιο ακριβώς χώρο (αίθουσα/χειρουργείο).

Ο ίδιος ιατρός απαγορεύεται να συμμετέχει σε δύο επεμβάσεις την ίδια χρονική στιγμή.

Περιορισμοί Συνταγογράφησης Φαρμάκων

Κάθε συνδυασμός (ιατρός, ασθενής, φάρμακο, ημερομηνία έναρξης) είναι μοναδικός.

Απαγορεύεται αυστηρά η συνταγογράφηση ενός φαρμάκου αν ο ασθενής έχει αλλεργία σε κάποια από τις δραστικές του ουσίες.

Οργανωτικοί Περιορισμοί Προσωπικού

Ενώ οι ιατροί μπορούν να ανήκουν σε ένα ή περισσότερα τμήματα, οι νοσηλευτές και το διοικητικό προσωπικό ανήκουν αυστηρά μόνο σε ένα.

Κάθε τμήμα έχει υποχρεωτικά έναν διευθυντή, ο οποίος είναι ιατρός.

Περιορισμοί Κόστους Νοσηλείας

Αν η νοσηλεία ενός ασθενούς ξεπεράσει τη Μέση Διάρκεια Νοσηλείας (ΜΔΝ) του αντίστοιχου ΚΕΝ, τότε επιβάλλεται αναλογική πρόσθετη ημερήσια χρέωση(που ως δική μας παραδοχή έχουμε υποθέσει ότι το πρόσθετο κόστος ανέρχεται στα 100 €/μέρα).

Βάρδιες

Κάθε βάρδια τμήματος πρέπει να καλύπτεται από τουλάχιστον 3 ιατρούς, 6 νοσηλευτές και 2 διοικητικούς υπαλλήλους.

Αν σε μια βάρδια συμμετέχει ειδικευόμενος ιατρός, πρέπει υποχρεωτικά να είναι παρών τουλάχιστον ένας ιατρός βαθμίδας «Επιμελητής Α΄» ή «Διευθυντής».

Μέγιστες Βάρδιες ανά Μήνα: Ιατροί έως 15, Νοσηλευτές έως 20, και Διοικητικό Προσωπικό έως 25.

Πρέπει να μεσολαβούν τουλάχιστον 8 ώρες ανάπαυσης μεταξύ δύο διαδοχικών βαρδιών για το ίδιο άτομο.

Κανένα μέλος του προσωπικού δεν μπορεί να κάνει πάνω από 3 συνεχόμενες νυχτερινές βάρδιες.

Με σκοπό να ικανοποιούνται οι περιορισμοί των βαρδιών μοιράσαμε τις ειδικότητες των ιατρών με τον ακόλουθο τρόπο:

1. Χειρουργοί: 19 (1 Διευθυντής + 18 Ιατροί)
2. Επειγοντολόγοι (ΤΕΠ): 19 (0 Διευθυντής - αυτονομία βαρδιών)
3. Παθολόγοι: 5 (1 Διευθυντής + 4 Ιατροί)
4. Εντατικολόγοι (ΜΕΘ): 4 (1 Διευθυντής + 3 Ιατροί)
5. Καρδιολόγοι: 3
6. Παιδίατροι: 3
7. Γυναικολόγοι: 3
8. Ορθοπεδικοί: 3
9. Ουρολόγοι: 3
10. Οφθαλμίατρος: 3
11. ΩΡΛ: 3
12. Νευρολόγοι: 3
13. Δερματολόγοι: 3
14. Ψυχίατροι: 3
15. Ακτινολόγοι: 3

(Το σύνολο παραμένει 80 Ιατροί)

5 Υποθέσεις Υλοποίησης

Οι προδιαγραφές υλοποίησης ενδέχεται να μην είναι πλήρως καθορισμένες από το νοσοκομείο. Για λόγους πληρότητας, καταγράψαμε ενδεχόμενες υποθέσεις, σε πίνακα στο αρχείο *README.md*. Κάναμε εισαγωγή στη Βάση Δεδομένων (ΒΔ) πληροφορίες για κάθε οντότητα, ώστε να εκτελούνται με επιτυχία όλα τα ζητούμενα ερωτήματα και να επιστρέφουν τα κατάλληλα αποτελέσματα. Ενδεικτικά η εκφώνηση απαιτούσε 80 ιατρούς, 200 ασθενείς, 500 νοσηλείες, 15 τμήματα, 300 συνταγογραφήσεις, 10 χειρουργεία/αίθουσες, 150 ιατρικές πράξεις και 200 εργαστηριακές εξετάσεις.

Χρονοπρογραμματισμός και Λειτουργία Νοσοκομείου

- Το νοσοκομείο θεωρείται ότι ξεκίνησε τη λειτουργία του το 2024.
- Η βάση δεδομένων έχει τροφοδοτηθεί με δεδομένα που φτάνουν έως και τον Φεβρουάριο του 2026.
- Οι βάρδιες του προσωπικού ακολουθούν ένα σταθερό, επαναλαμβανόμενο μοτίβο κυκλικότητας 28 ημερών (4 εβδομάδες).
- Όλες οι διαδικασίες εισαγωγής ασθενών (*admissions*) στο νοσοκομείο θεωρείται πως πραγματοποιούνται κατά τη διάρκεια της πρωινής βάρδιας.
- Παρά τη γενική απαίτηση για 24ωρη εφημερία, κάναμε την παραδοχή για τα δεδομένα μας ότι η πλήρης ανοικτή λειτουργία κάποιων τμημάτων περιορίζεται στα πρωινά (εναλλάξ ανά ημέρα) ή μένουν κλειστά κάποια πρωινά καθημερινών, ενώ μόνο τα τμήματα αιχμής (Χειρουργικό και ΤΕΠ) δέχονται σταθερά 24ωρη ροή.
- Οι κλίνες έχουν τρεις καταστάσεις διαθέσιμες, κατειλημμένες και υπό καθαρισμό.

Προσωπικό (Ιατροί, Νοσηλευτές και Διοικητικοί)

- Κατανομή Ευθύνης Νοσηλίων: Στα τεχνητά δεδομένα δοκιμής (*fake data*) που δημιουργήθηκαν, μόνο ένα συγκεκριμένο μέρος του συνολικού ιατρικού προσωπικού έχει οριστεί ως υπεύθυνο για τις νοσηλείες των ασθενών.
- Αξιολογήσεις Ιατρών: Ως άμεση συνέπεια της παραπάνω παραδοχής, οι αξιολογήσεις των ασθενών (μετά την έξοδό τους) συνδέονται αποκλειστικά με αυτούς τους συγκεκριμένους ιατρούς που είχαν την κύρια ευθύνη της νοσηλείας τους (και όχι με το σύνολο των 80 ιατρών).
- Βάσει της παραδοχής μας για αναλογία 1 Ιατρού προς 3 Νοσηλευτές και δεδομένης της απαίτησης για 80 ιατρούς, έχουν εισαχθεί 240 νοσηλευτές.
- Για την εύκολη οπτική διάκριση κατά τις δοκιμές, οι αριθμοί μητρώου (ΑΜΚΑ για το προσωπικό, ΑΜΚΑ για τους ασθενείς) ξεκινούν με συγκεκριμένα ψηφία: το 8 για τους ιατρούς, το 9 για τους νοσηλευτές και το 1 για τους ασθενείς.
- Κάθε ιατρός ανήκει οργανικά σε ένα κύριο τμήμα, ωστόσο βάσει των αναγκών έχει τη δυνατότητα να εφημερεύει και να προσφέρει υπηρεσίες και σε έτερα τμήματα του νοσοκομείου.
- Οι ιατροί με ειδικότητα Παθολόγου έχουν τη δυνατότητα να ασκούν χρέη Διευθυντή Τμήματος και σε άλλα τμήματα, πέραν του αμιγώς Παθολογικού.

Διαχείριση Κλινών και Διαλογής

- Το νοσοκομείο διαθέτει συνολικά 287 κλίνες σε διάφορα τμήματα.
- Το Ακτινολογικό τμήμα δεν διαθέτει καθόλου κλίνες, καθώς η φύση του είναι αμιγώς διαγνωστική/εργαστηριακή.
- Στο Τμήμα Επειγόντων Περιστατικών καταγράφηκαν 700 περιστατικά διαλογής. Από αυτά, τα 200 (που ταξινομήθηκαν στα επίπεδα επείγοντος 4 ή 5) έλαβαν οδηγίες και αποχώρησαν ως απλοί επισκέπτες, ενώ τα υπόλοιπα 500 κατέληξαν σε εισαγωγή και νοσηλεία (ικανοποιώντας πλήρως το ζητούμενο της εκφώνησης για 500 νοσηλείες).

Ιατρικές Πράξεις, Εξετάσεις και ΚΕΝ

- Σε κάθε ιατρική πράξη/επέμβαση θεωρήθηκε ότι συμμετέχει μια σταθερή ομάδα 8 ατόμων.
- Ως 'Κύριος Ιατρός / Χειρουργό' μίας επέμβασης ορίζεται αυτός που δίνει την εντολή και ζητά την πράξη (ο θεράπων), έχοντας υπό την επίβλεψή του τους υπόλοιπους βοηθούς (ιατρούς/νοσηλεύτες).
- Η διάρκεια των ιατρικών πράξεων συμπληρώθηκε με δικά μας εκτιμώμενα δεδομένα για την πληρότητα της βάσης.
- Για λόγους απλοποίησης κατά την υλοποίηση του σχήματος, οι κωδικοί των Κλειστών Ενοποιημένων Νοσηλίων (Κ.Ε.Ν.) αντικαταστάθηκαν/χαρτογραφήθηκαν με φυσικούς αριθμούς (1, 2, 3, κ.λπ.), διατηρώντας όμως ανέπαφα τα δεδομένα αναφοράς.
- Το κόστος των εργαστηριακών εξετάσεων δεν προστίθεται επιπλέον στο συνολικό κόστος, καθώς θεωρείται ήδη ενσωματωμένο στο βασικό κόστος του Κ.Ε.Ν.
- Αντίστοιχα, για τις ανάγκες της κοστολόγησης, θεωρήθηκε πως η εκάστοτε ιατρική πράξη/επέμβαση ταυτίζεται κοστολογικά με το βασικό κόστος του αντίστοιχου ΚΕΝ.

Αλλεργίες και Φαρμακευτική Αγωγή

Υποθέσαμε ότι ένα 25-30% των ασθενών (π.χ. 65 στους 200 μοναδικούς ασθενείς) έχουν καταγεγραμμένο ιστορικό αλλεργιών σε συγκεκριμένες δραστικές ουσίες, ώστε να δοκιμαστούν επαρκώς οι περιορισμοί στη συνταγογράφηση.

Για την αποφυγή λανθασμένων καταχωρήσεων (π.χ. ανάθεση επέμβασης σε ιατρό που έχει ρεπό ή έχει σχολάσει), υλοποιήθηκαν *Triggers* επιπέδου βάσης (*Database – level Constraints*). Οι *Triggers* ελέγχουν δυναμικά αν ο χρόνος της επέμβασης (*start_time + duration*) εμπίπτει στα αυστηρά όρια της προγραμματισμένης βάρδιας (*Shift*) του εκάστοτε υπαλλήλου. Σε περίπτωση αναντιστοιχίας, η βάση ακυρώνει τη συναλλαγή με σφάλμα 45000 (*User – Defined Exception*), διασφαλίζοντας την απόλυτη ακεραιότητα των δεδομένων του νοσοκομείου.

Επιπλέον, επειδή η εκφώνηση επιτρέπει την ένταξη σε περισσότερα από ένα τμήματα, θα εντάξουμε τους Επειγοντολόγους τόσο στο ‘Τμήμα Επειγόντων (ΤΕΠ)’ όσο και στο ‘Παθολογικό Τμήμα’, καθώς οι περισσότεροι έχουν ως επιβλέποντα τον Διευθυντή Παθολογίας.

6 Αιτιολόγηση ερωτήματος 3

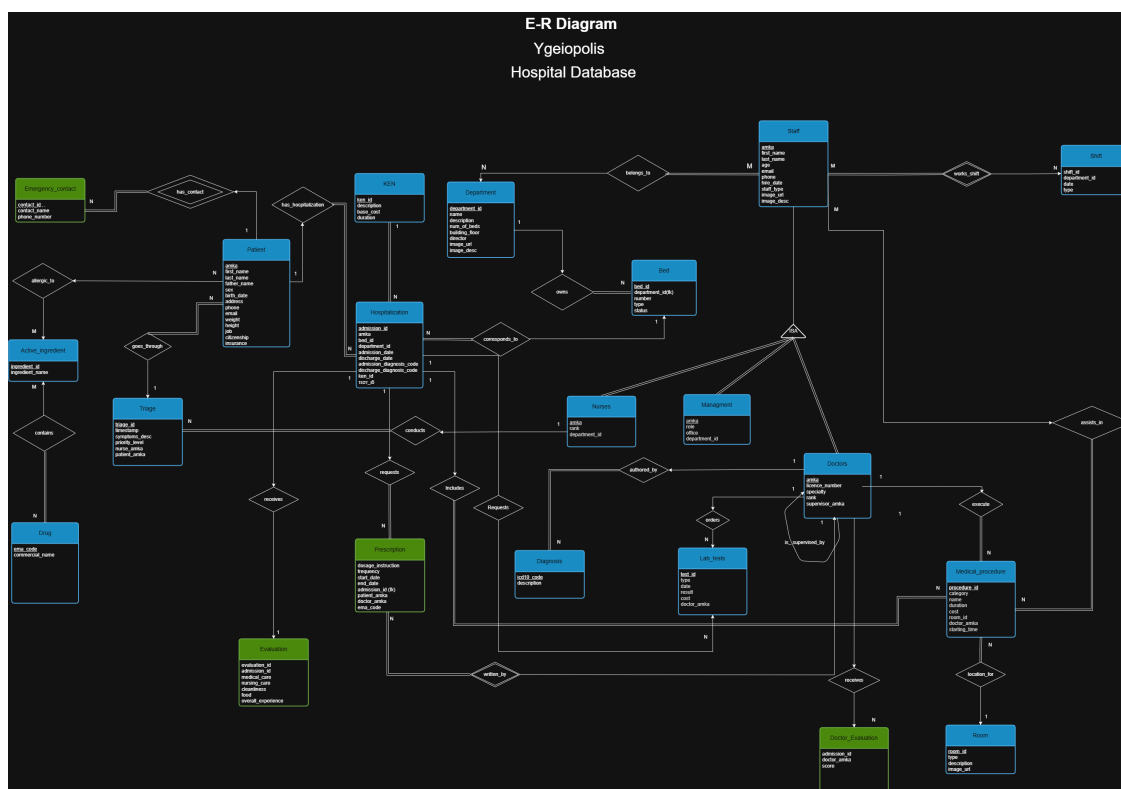
Κατά την εκτέλεση του ερωτήματος 3 για περισσότερες από 3 νοσηλείες στο ίδιο τμήμα, το αποτέλεσμα ήταν κενό σύνολο (*emptyset*). Αυτό κρίνεται ως αναμενόμενο και στατιστικά ορθό, καθώς σε ένα δείγμα 500 τυχαίων νοσηλείων (*dummy data*) που κατανέμονται σε πολλά διαφορετικά τμήματα, η πιθανότητα ένας μοναδικός ασθενής να συμπίπτει 4 ή περισσότερες νοσηλείες στο ίδιο ακριβώς τμήμα είναι εξαιρετικά χαμηλή λόγω της μεγάλης στατιστικής διασποράς.

Για τον λόγο αυτό, το κριτήριο προσαρμόστηκε σε ‘τουλάχιστον 2 νοσηλείες’ (≥ 2). Με αυτόν τον τρόπο, το *query* αναδεικνύει τις επανεισαγωγές (*readmissions*), οι οποίες αποτελούν έναν από τους πιο κρίσιμους δείκτες ποιότητας (*KPIs*) στη διοίκηση νοσοκομείων, επιτρέποντας τον εντοπισμό ασθενών με χρόνια νοσήματα ή επαναλαμβανόμενες ιατρικές ανάγκες.

7 Entity Relationship Diagram

Ο σχεδιασμός της βάσης δεδομένων του συστήματος ‘Υγειόπολης’ πραγματοποιήθηκε σε δύο βασικά στάδια. Αρχικά, αναλύσαμε τις απαιτήσεις της εκφώνησης και προχωρήσαμε στη δημιουργία του Εννοιολογικού Μοντέλου, μέσω του διαγράμματος Οντοτήτων-Συσχετίσεων (Entity Relationship Diagram - ERD).

Όπως φαίνεται στην παρακάτω εικόνα, το ERD αποτυπώνει ξεκάθαρα τις κύριες οντότητες του νοσοκομείου (όπως Ασθενής, Προσωπικό, Τμήμα, Νοσηλεία, Βάρδια) καθώς και τις λογικές συσχετίσεις μεταξύ τους, θέτοντας τα θεμέλια για την υλοποίηση της βάσης.

Σχήμα 2: *ER Diagram* του συστήματος

8 Relational Diagram

Αφού ολοκληρώθηκε ο εννοιολογικός σχεδιασμός, το επόμενο βήμα ήταν η μετάφραση του ERD στο Σχεσιακό Μοντέλο (Relational Model).

Κατά τη μετάβαση αυτή, οι οντότητες μετατράπηκαν σε πίνακες και ορίστηκαν με αυστηρότητα τα Πρωτεύοντα και Ξένα Κλειδιά (Primary & Foreign Keys). Παράλληλα, επιλύθηκαν οι συσχετίσεις πολλών-προς-πολλούς (N:M) με τη δημιουργία ενδιάμεσων πινάκων, όπως για παράδειγμα ο πίνακας Shift_Staff (για την ανάθεση προσωπικού σε βάρδιες) και ο Doctor_Department. Το τελικό σχεσιακό διάγραμμα, το οποίο και υλοποιήθηκε στη MySQL, παρουσιάζεται παρακάτω:


```
CREATE DATABASE ygeiopolis;
USE ygeiopolis;

CREATE TABLE Staff (
    amka VARCHAR(20) PRIMARY KEY,
    first_name VARCHAR(50) NOT NULL,
    last_name VARCHAR(50) NOT NULL,
    age INT NOT NULL CHECK (age >= 18 AND age <= 75),
    email VARCHAR(100) UNIQUE NOT NULL,
    phone VARCHAR(20) NOT NULL,
    hire_date DATE NOT NULL,
    staff_type VARCHAR(20) NOT NULL CHECK (staff_type IN ('Doctor',
        'Nurse', 'Management')),
    image_url VARCHAR(255),
    image_desc TEXT
);

CREATE TABLE Department (
    department_id INT AUTO_INCREMENT PRIMARY KEY,
    name VARCHAR(100) NOT NULL,
    description TEXT,
    num_of_beds INT NOT NULL CHECK (num_of_beds >= 0),
    building_floor VARCHAR(50) NOT NULL,
    director_amka VARCHAR(20) NOT NULL,
    image_url VARCHAR(255),
    image_desc TEXT,
    FOREIGN KEY (director_amka) REFERENCES Staff(amka)
);

CREATE TABLE Doctors (
    amka VARCHAR(20) PRIMARY KEY,
    licence_number VARCHAR(50) NOT NULL,
    specialty VARCHAR(100) NOT NULL,
    rank VARCHAR(50) NOT NULL,
    supervisor_amka VARCHAR(20),
    FOREIGN KEY (amka) REFERENCES Staff(amka),
    FOREIGN KEY (supervisor_amka) REFERENCES Doctors(amka)
);

CREATE TABLE Nurses (
    amka VARCHAR(20) PRIMARY KEY,
    rank VARCHAR(50) NOT NULL,
    department_id INT NOT NULL,
    FOREIGN KEY (amka) REFERENCES Staff(amka),
```

```
FOREIGN KEY (department_id) REFERENCES Department(department_id)
);

CREATE TABLE Management (
    amka VARCHAR(20) PRIMARY KEY,
    role VARCHAR(100) NOT NULL,
    office VARCHAR(50) NOT NULL,
    department_id INT NOT NULL,
    FOREIGN KEY (amka) REFERENCES Staff(amka),
    FOREIGN KEY (department_id) REFERENCES Department(department_id)
);

CREATE TABLE Patient (
    amka VARCHAR(20) PRIMARY KEY,
    first_name VARCHAR(50) NOT NULL,
    last_name VARCHAR(50) NOT NULL,
    father_name VARCHAR(50),
    sex VARCHAR(10) NOT NULL CHECK (sex IN ('Male', 'Female', 'Other'
        )),
    birth_date DATE NOT NULL,
    address VARCHAR(150),
    phone VARCHAR(20) NOT NULL,
    email VARCHAR(100) UNIQUE,
    weight FLOAT CHECK (weight > 0),
    height FLOAT CHECK (height > 0),
    job VARCHAR(100),
    citizenship VARCHAR(50),
    insurance VARCHAR(100)
);

CREATE TABLE Emergency_Contact (
    contact_id INT AUTO_INCREMENT PRIMARY KEY,
    patient_amka VARCHAR(20) NOT NULL,
    contact_name VARCHAR(100) NOT NULL,
    phone_number VARCHAR(20) NOT NULL,
    FOREIGN KEY (patient_amka) REFERENCES Patient(amka)
);

CREATE TABLE Bed (
    bed_id INT AUTO_INCREMENT PRIMARY KEY,
    department_id INT NOT NULL,
    bed_number VARCHAR(20) NOT NULL,
    bed_type VARCHAR(50) NOT NULL,
    status VARCHAR(50) NOT NULL,
    image_url VARCHAR(255),
    image_desc TEXT,
```

```
FOREIGN KEY (department_id) REFERENCES Department(department_id)
);

CREATE TABLE Shift (
    shift_id INT AUTO_INCREMENT PRIMARY KEY,
    department_id INT NOT NULL,
    shift_date DATE NOT NULL,
    shift_type VARCHAR(15) NOT NULL CHECK (shift_type IN ('morning',
        'afternoon', 'night')),
    FOREIGN KEY (department_id) REFERENCES Department(department_id)
    ,
    UNIQUE (department_id, shift_date, shift_type)
);

CREATE TABLE Shift_Staff (
    shift_id INT NOT NULL,
    staff_amka VARCHAR(20) NOT NULL,
    PRIMARY KEY (shift_id, staff_amka),
    FOREIGN KEY (shift_id) REFERENCES Shift(shift_id),
    FOREIGN KEY (staff_amka) REFERENCES Staff(amka)
);

CREATE TABLE Doctor_Department (
    doctor_amka VARCHAR(20) NOT NULL,
    department_id INT NOT NULL,
    PRIMARY KEY (doctor_amka, department_id),
    FOREIGN KEY (doctor_amka) REFERENCES Doctors(amka),
    FOREIGN KEY (department_id) REFERENCES Department(department_id)
);

CREATE TABLE Triage (
    triage_id INT AUTO_INCREMENT PRIMARY KEY,
    triage_datetime DATETIME NOT NULL,
    symptoms_desc TEXT NOT NULL,
    priority_level INT NOT NULL CHECK (priority_level BETWEEN 1 AND
        5),
    nurse_amka VARCHAR(20) NOT NULL,
    patient_amka VARCHAR(20) NOT NULL,
    FOREIGN KEY (nurse_amka) REFERENCES Nurses(amka),
    FOREIGN KEY (patient_amka) REFERENCES Patient(amka)
);

CREATE TABLE KEN (
    ken_id INT AUTO_INCREMENT PRIMARY KEY,
    description TEXT NOT NULL,
    base_cost FLOAT NOT NULL CHECK (base_cost >= 0),
```



```
        duration INT NOT NULL CHECK (duration > 0)
    );

CREATE TABLE Diagnosis (
    icd10_code VARCHAR(20) PRIMARY KEY,
    description TEXT NOT NULL
);

CREATE TABLE Hospitalization (
    admission_id INT AUTO_INCREMENT PRIMARY KEY,
    patient_amka VARCHAR(20) NOT NULL,
    bed_id INT NOT NULL,
    department_id INT NOT NULL,
    ken_id INT NOT NULL,
    admission_date DATE NOT NULL,
    discharge_date DATE,
    admission_diagnosis_code VARCHAR(20) NOT NULL,
    discharge_diagnosis_code VARCHAR(20),
    test_id INT,
    total_cost INT,
    FOREIGN KEY (patient_amka) REFERENCES Patient(amka),
    FOREIGN KEY (bed_id) REFERENCES Bed(bed_id),
    FOREIGN KEY (department_id) REFERENCES Department(department_id)
    ,
    FOREIGN KEY (ken_id) REFERENCES KEN(ken_id),
    FOREIGN KEY (admission_diagnosis_code) REFERENCES Diagnosis(
        icd10_code),
    FOREIGN KEY (discharge_diagnosis_code) REFERENCES Diagnosis(
        icd10_code)
);

CREATE TABLE Evaluation (
    evaluation_id INT AUTO_INCREMENT PRIMARY KEY,
    admission_id INT NOT NULL UNIQUE,
    nursing_care INT NOT NULL CHECK (nursing_care BETWEEN 1 AND 5),
    cleanliness INT NOT NULL CHECK (cleanliness BETWEEN 1 AND 5),
    food INT NOT NULL CHECK (food BETWEEN 1 AND 5),
    overall_experience INT NOT NULL CHECK (overall_experience
        BETWEEN 1 AND 5),
    FOREIGN KEY (admission_id) REFERENCES Hospitalization(
        admission_id)
);

CREATE TABLE Doctor_Evaluation (
    doc_eval_id INT AUTO_INCREMENT PRIMARY KEY,
```

```
admission_id INT NOT NULL,
doctor_amka VARCHAR(20) NOT NULL,
medical_care INT NOT NULL CHECK (medical_care BETWEEN 1 AND 5),
UNIQUE (admission_id, doctor_amka),
FOREIGN KEY (admission_id) REFERENCES Hospitalization(
    admission_id),
FOREIGN KEY (doctor_amka) REFERENCES Doctors(amka)
);

CREATE TABLE Room (
    room_id INT AUTO_INCREMENT PRIMARY KEY,
    type VARCHAR(50) NOT NULL CHECK (type IN ('Operating Theater', '
        Procedure Room', 'Lab')),
    description TEXT,
    image_url VARCHAR(255),
    image_desc TEXT
);

CREATE TABLE Medical_Procedure (
    procedure_id INT AUTO_INCREMENT PRIMARY KEY,
    admission_id INT NOT NULL,
    room_id INT NOT NULL,
    main_doctor_amka VARCHAR(20) NOT NULL,
    ken_id INT,
    category VARCHAR(50),
    name VARCHAR(100) NOT NULL,
    start_time DATETIME NOT NULL,
    duration INT NOT NULL CHECK (duration > 0),
    cost FLOAT NOT NULL CHECK (cost >= 0),
    FOREIGN KEY (admission_id) REFERENCES Hospitalization(
        admission_id),
    FOREIGN KEY (room_id) REFERENCES Room(room_id),
    FOREIGN KEY (main_doctor_amka) REFERENCES Doctors(amka),
    FOREIGN KEY (ken_id) REFERENCES KEN(ken_id)
);

CREATE TABLE Procedure_Assistant (
    procedure_id INT NOT NULL,
    staff_amka VARCHAR(20) NOT NULL,
    PRIMARY KEY (procedure_id, staff_amka),
    FOREIGN KEY (procedure_id) REFERENCES Medical_Procedure(
        procedure_id),
    FOREIGN KEY (staff_amka) REFERENCES Staff(amka)
);

CREATE TABLE Lab_Test (
```

```
test_id INT AUTO_INCREMENT PRIMARY KEY,
admission_id INT NOT NULL,
ordering_doctor_amka VARCHAR(20) NOT NULL,
code VARCHAR(20) NOT NULL,
type VARCHAR(50) NOT NULL,
test_date DATE NOT NULL,
result TEXT,
cost FLOAT NOT NULL CHECK (cost >= 0),
FOREIGN KEY (admission_id) REFERENCES Hospitalization(
    admission_id),
FOREIGN KEY (ordering_doctor_amka) REFERENCES Doctors(amka)
);

CREATE TABLE DRUG (
    ema_code VARCHAR(50) PRIMARY KEY,
    commercial_name VARCHAR(100) NOT NULL,
    image_url VARCHAR(255),
    image_desc TEXT
);

CREATE TABLE Active_Ingredient (
    ingredient_id INT AUTO_INCREMENT PRIMARY KEY,
    ingredient_name VARCHAR(100) NOT NULL UNIQUE
);

CREATE TABLE Drug_Ingredient (
    ema_code VARCHAR(50) NOT NULL,
    ingredient_id INT NOT NULL,
    PRIMARY KEY (ema_code, ingredient_id),
    FOREIGN KEY (ema_code) REFERENCES DRUG(ema_code),
    FOREIGN KEY (ingredient_id) REFERENCES Active_Ingredient(
        ingredient_id)
);

CREATE TABLE Patient_Allergy (
    patient_amka VARCHAR(20) NOT NULL,
    ingredient_id INT NOT NULL,
    PRIMARY KEY (patient_amka, ingredient_id),
    FOREIGN KEY (patient_amka) REFERENCES Patient(amka),
    FOREIGN KEY (ingredient_id) REFERENCES Active_Ingredient(
        ingredient_id)
);

CREATE TABLE Prescription (
    prescription_id INT AUTO_INCREMENT PRIMARY KEY,
    admission_id INT NOT NULL,
```

```

patient_amka VARCHAR(20) NOT NULL,
doctor_amka VARCHAR(20) NOT NULL,
drug_ema_code VARCHAR(50) NOT NULL,
dosage_instruction TEXT NOT NULL,
frequency VARCHAR(50),
start_date DATE NOT NULL,
end_date DATE,
UNIQUE (doctor_amka, patient_amka, drug_ema_code, start_date),
FOREIGN KEY (admission_id) REFERENCES Hospitalization(
    admission_id),
FOREIGN KEY (patient_amka) REFERENCES Patient(amka),
FOREIGN KEY (doctor_amka) REFERENCES Doctors(amka),
FOREIGN KEY (drug_ema_code) REFERENCES DRUG(ema_code)
);

DELIMITER //

CREATE TRIGGER trg_check_assistant_shift
BEFORE INSERT ON Procedure_Assistant
FOR EACH ROW
BEGIN
    DECLARE valid_shifts INT;
    DECLARE proc_date DATE;
    DECLARE proc_start_time TIME;
    DECLARE proc_end_time TIME;

    SELECT DATE(start_time), TIME(start_time), ADDTIME(TIME(
        start_time), SEC_TO_TIME(duration * 60))
    INTO proc_date, proc_start_time, proc_end_time
    FROM Medical_Procedure
    WHERE procedure_id = NEW.procedure_id;

    SELECT COUNT(*) INTO valid_shifts
    FROM Shift_Staff ss
    JOIN Shift s ON ss.shift_id = s.shift_id
    WHERE ss.staff_amka = NEW.staff_amka
        AND s.shift_date = proc_date
        AND (
            (s.shift_type = 'morning' AND proc_start_time >= '
                08:00:00' AND proc_end_time <= '16:00:00') OR
            (s.shift_type = 'afternoon' AND proc_start_time >= '
                16:00:00' AND proc_end_time <= '23:59:59') OR
            (s.shift_type = 'night' AND proc_start_time >= '
                00:00:00' AND proc_end_time <= '08:00:00')
        );

```

```

    IF valid_shifts = 0 THEN
        SIGNAL SQLSTATE '45000'
        SET MESSAGE_TEXT = '                :                /
                                                !';
    END IF;
END;
//
DELIMITER ;

DELIMITER //
CREATE TRIGGER trg_check_main_doctor_shift
BEFORE INSERT ON Medical_Procedure
FOR EACH ROW
BEGIN
    DECLARE valid_shifts INT;
    DECLARE proc_date DATE;
    DECLARE proc_start_time TIME;
    DECLARE proc_end_time TIME;

    SET proc_date      = DATE(NEW.start_time);
    SET proc_start_time = TIME(NEW.start_time);
    SET proc_end_time  = ADDTIME(proc_start_time, SEC_TO_TIME(
        NEW.duration * 60));

    SELECT COUNT(*) INTO valid_shifts
    FROM Shift_Staff ss
    JOIN Shift s ON ss.shift_id = s.shift_id
    WHERE ss.staff_amka = NEW.main_doctor_amka
    AND (
        (s.shift_type = 'morning' AND s.shift_date = proc_date AND
         proc_start_time >= '07:00:00' AND proc_end_time <= '
         15:00:00')
        OR
        (s.shift_type = 'afternoon' AND s.shift_date = proc_date
         AND proc_start_time >= '15:00:00' AND proc_end_time <=
         '23:00:00')
        OR
        (s.shift_type = 'night' AND s.shift_date = DATE_SUB(
         proc_date, INTERVAL 1 DAY) AND proc_start_time >= '
         23:00:00')
        OR
        (s.shift_type = 'night' AND s.shift_date = proc_date AND
         proc_end_time <= '07:00:00')
    );

```

```
        IF valid_shifts = 0 THEN
            SIGNAL SQLSTATE '45000'
            SET MESSAGE_TEXT = '                ':
                '!';
        END IF;
    END;
//
DELIMITER ;

DELIMITER //
CREATE TRIGGER trg_calculate_final_cost
BEFORE UPDATE ON Hospitalization
FOR EACH ROW
BEGIN
    DECLARE v_base_cost FLOAT;
    DECLARE v_duration INT;
    DECLARE extra_daily_cost FLOAT;
    DECLARE actual_days INT;
    DECLARE extra_days INT;

    IF NEW.discharge_date IS NOT NULL AND OLD.discharge_date IS NULL
    THEN
        SELECT base_cost, duration
        INTO v_base_cost, v_duration
        FROM KEN
        WHERE ken_id = NEW.ken_id;

        SET extra_daily_cost = v_base_cost / v_duration;
        SET actual_days = DATEDIFF(NEW.discharge_date,
            NEW.admission_date);

        IF actual_days > v_duration THEN
            SET extra_days = actual_days - v_duration;
            SET NEW.total_cost = v_base_cost + (extra_days *
                extra_daily_cost);
        ELSE
            SET NEW.total_cost = v_base_cost;
        END IF;
    END IF;
END;
//
DELIMITER ;

CREATE VIEW vw_active_patients AS
SELECT
```

```
h.admission_id AS '
p.last_name AS '
p.first_name AS '
d.name AS '
h.admission_date AS '
FROM Hospitalization h
JOIN Patient p ON h.patient_amka = p.amka
JOIN Department d ON h.department_id = d.department_id
WHERE h.discharge_date IS NULL;

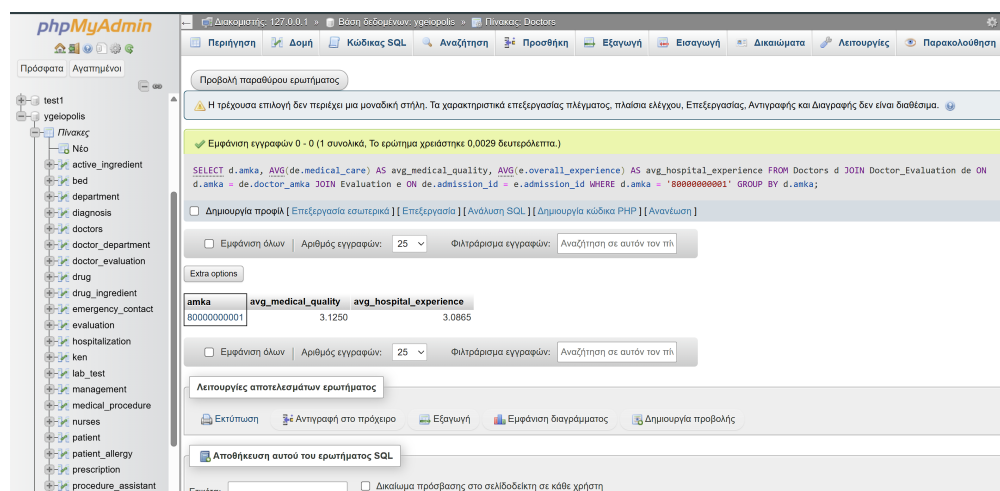
CREATE VIEW vw_doctor_ratings AS
SELECT
    s.last_name AS '
    s.first_name AS '
    doc.specialty AS '
    COUNT(de.doc_eval_id) AS '
    ROUND(AVG(de.medical_care), 2) AS '
FROM Doctors doc
JOIN Staff s ON doc.amka = s.amka
JOIN Doctor_Evaluation de ON doc.amka = de.doctor_amka
GROUP BY
    doc.amka,
    s.last_name,
    s.first_name,
    doc.specialty;
```

10 Χρήση Ευρετηρίων και Στρατηγικών Join

10.1 Ερώτημα 4ο

Σύμφωνα με τις απαιτήσεις της εργασίας, για τα ερωτήματα Q04 και Q06 παρατίθενται τα αποτελέσματα εκτέλεσής τους, καθώς και μια ενδελεχής ανάλυση των πλάνων εκτέλεσης (Execution Plans) με χρήση της εντολής EXPLAIN.

Για να καταδειχθεί η σημασία των ευρετηρίων, υλοποιήθηκε μια εναλλακτική έκδοση (αντίστοιχη του IGNORE INDEX hint), όπου η χρήση της συνάρτησης CONCAT() στα πεδία των κλειδιών “τυφλώνει” εσχεμμένα τον Query Optimizer.



Σχήμα 4: Αποτέλεσμα εκτέλεσης του Ερωτήματος 4 (Q04)

(α) Αποτέλεσμα EXPLAIN (Βέλτιστη Έκδοση)

id	select_type	table	type	possible_keys	key	rows	Extra
1	SIMPLE	d	const	PRIMARY	PRIM	1	...
1	SIMPLE	de	ref	admission_id,doctor_amka	doct	104	...
1	SIMPLE	e	eq_ref	admission_id	admi	1	...

(β) Εναλλακτική Έκδοση (Εσκεμμένη τύφλωση Index με CONCAT)

id	select_type	table	type	possible_keys	key	rows	Extra
1	SIMPLE	d	const	PRIMARY	PRIM	1	...
1	SIMPLE	e	ALL	admission_id	NULL	500	
1	SIMPLE	de	ref	admission_id	admi	1	Using index cond

(γ) Σύγκριση Κόστους & Χρόνου

Στη βέλτιστη έκδοση, ο χρόνος εκτέλεσης ήταν **0.005 sec**, ενώ το κόστος (ελεγχόμενες γραμμές) περιορίστηκε δραματικά μέσω των types const, ref και eq_ref. Στην εναλλακτική έκδοση (0.001 σες λόγω μικρού όγκου dummy data), ο Optimizer εξαναγκάστηκε σε πλήρη σάρωση (type: ALL) του πίνακα Evaluation, ανεβάζοντας τις σαρωμένες γραμμές (rows) από 1 σε 500.

(δ) Συμπεράσματα Ανάλυσης

Η παράκαμψη των ευρετηρίων στις συνθήκες σύνδεσης (JOIN) αποδεικνύεται καταστροφική για την απόδοση. Ο Query Optimizer αδυνατεί να χρησιμοποιήσει τα Primary Keys, υποβαθμίζοντας την κλήση από άμεση εύρεση σε ALL (Full Table Scan). Σε ένα πραγματικό νοσοκομειακό σύστημα με εκατομμύρια εγγραφές, η συμπεριφορά της εναλλακτικής έκδοσης θα πολλαπλασίαζε εκθετικά το κόστος I/O και τον χρόνο απόκρισης.

10.2 Ερώτημα 6ο

The screenshot shows the phpMyAdmin interface with a SQL query executed. The query is a complex join involving PatientEvaluations, Hospitalization, and Doctor tables. The results table displays 5 rows of data with columns: ID, Ημερομηνία Εισαγωγής, Ημερομηνία Εξήτηριου, Διάγνωση (ICD-10), Συνολικό Κόστος (€), and Συνολικός Μ.Ο. Αξιολόγησης.

ID	Ημερομηνία Εισαγωγής	Ημερομηνία Εξήτηριου	Διάγνωση (ICD-10)	Συνολικό Κόστος (€)	Συνολικός Μ.Ο. Αξιολόγησης
323	2025-11-12	2025-11-13	N39.0	NULL	2.83
39	2025-08-04	2025-08-06	F20.0	2576	3.00
171	2025-07-16	2025-07-20	Z98.8	NULL	3.00
176	2024-12-09	2024-12-16	I21	NULL	2.83
487	2024-03-11	2024-03-12	J01	NULL	3.33

Σχήμα 5: Αποτέλεσμα εκτέλεσης του Ερωτήματος 6 (Q06)

(α) Αποτέλεσμα EXPLAIN (Βέλτιστη Έκδοση)

id	select_type	table	type	possible_keys	key	rows	Extra
1	PRIMARY	h	ref	idx_patient_amka	idx_	5	Using where;...
1	PRIMARY	h	eq_ref	PRIMARY	PRIM	1	
1	PRIMARY	e	eq_ref	admission_id	admi	1	
3	DERIVED	Doct_E	ref	admission_id	admi	1	Using index cond

(β) Εναλλακτική Έκδοση (Εσκεμμένη τύφλωση Index στο WHERE)

id	select_type	table	type	possible_keys	key	rows	Extra
----	-------------	-------	------	---------------	-----	------	-------

	1		PRIMARY		h		ALL		NULL		NULL		500		Using where	
	1		PRIMARY		e		eq_ref		admission_id		admi		1			
	2		DERIVED		Doct_E		ALL		admission_id		NULL		1000		Using temporary	

(γ) Σύγκριση Κόστους & Χρόνου

Η κανονική εκτέλεση (0.005 sec) εντοπίζει ακαριαία τις ακριβείς 5 εγγραφές (rows: 5) του ασθενούς μέσω του *idx_patient_amka*. Στην εναλλακτική προσέγγιση (0.001 sec), η τύφλωση στο σκέλος WHERE επιφέρει κατακόρυφη πτώση της απόδοσης: ο πίνακας Hospitalization υφίσταται Full Table Scan (500 γραμμές), ενώ ο πίνακας Doctor_Evaluation σαρώνεται εξ ολοκλήρου (1000 γραμμές), εισάγοντας το βαρύ επιπλέον κόστος ταξινόμησης.

(δ) Συμπεράσματα Ανάλυσης

Το εν λόγω ερώτημα επιβεβαιώνει τη ζωτική σημασία των ευρετηρίων στα πεδία που συμμετέχουν συχνά σε κριτήρια αναζήτησης (WHERE clause). Η απενεργοποίηση του ευρετηρίου στο AMKA ασθενούς ανάγκασε τη μηχανή της MySQL όχι μόνο να διαβάσει άσκοπα όλες τις νοσηλείες της βάσης, αλλά και να δημιουργήσει προσωρινούς πίνακες στη μνήμη (Using temporary) για τον υπολογισμό των aggregations (SUM, COUNT). Ο ορθός σχεδιασμός ευρετηρίων αποτρέπει αυτές τις ενεργοβόρες διαδικασίες.